PA2 Report

Task 1A – IP-stride prefetcher understanding:-

The IP-stride prefetcher learns one constant stride per load instruction (IP) and prefetches 3 lines ahead once the same stride is seen twice in a row.

- **Table:** 24 fully associative trackers, each holding ip (low 8 bits only, MASK_IP 0xFF), last_cl_addr, last_stride and an lru rank. The 8-bit IP tag saves storage but lets different IPs alias onto one tracker.
- **Lookup / allocation:** search all 24 trackers for the IP; on a miss, replace the tracker with lru == 23 and return without prefetching (no history yet).
- **Stride:** computed on cache-line addresses (addr >> 6). The subtraction is done on the larger minus the smaller and the sign flipped, because unsigned subtraction would wrap; this makes negative strides work.
- **Confidence:** implicit and 1-bit: prefetch only when stride == last_stride.
- **Issue:** prefetch cl + stride·i for i = 1…3, stopping (break) at the 4 KB page boundary since later targets would be further outside. If the MSHR is more than half full, fill into L2 instead of L1 to avoid blocking demand misses.
- **Update:** store the new address and stride, and move the tracker to LRU rank 0.

Limitation (we make it better in spp): with only one stride remembered, a repeating pattern such as (+1, +2, +2, +2, +3) is caught only in the middle of the +2 run, giving about 50% accuracy and about 20% coverage on that pattern.

Task 1B – Stride Pattern Prefetcher (SPP)

Design

- **IP Table (IPT), 64 entries, direct-mapped by ip % 64:** per-IP state. Works on the line offset inside the 4 KB page (0–63), so a stride is offset − last_offset and a page change resets the entry's history.
- **Uniform detection first:** a 2-bit stride_conf goes up when the stride repeats and down when it changes. At stride_conf ≥ 2 the IP is treated as uniform and prefetches offset + stride·i like IP-stride.
- **Non-uniform fallback (signature):** each IP keeps a 12-bit signature of its recent strides, updated as sig = ((sig << 3) XOR (stride & 0x7F)) & 0xFFF. The & 0x7F keeps the 7-bit

two's-complement form of negative strides so their sign bits do not flip the history. Roughly the last 4 strides survive in 12 bits.

- **Pattern Table (PT), 4096 entries indexed by signature:** stores the stride that followed that history plus a 2-bit confidence. Training writes PT[old sig] before the signature is updated, so the row maps "this history" to "the stride that came next".
- **Lookahead for degree > 1:** predict a stride from PT[sig], prefetch it, update a speculative signature with the predicted stride, and repeat. The chain stops at the degree, at PT confidence < 1, or at the page boundary.
- **Training only on demand accesses:** type == PREFETCH calls are ignored.

Accuracy-based throttling

Every 256 prefetches issued to the lower level, accuracy is measured over that window only, and the degree is set:

Window accuracy	Degree	Why
≥ 75%	6	predictions are reliable; run further ahead to hide more latency
50–75%	4	mostly right; moderate lookahead
25–50%	2	half the prefetches are wasted; limit pollution
< 25%	1	keep a single probe prefetch

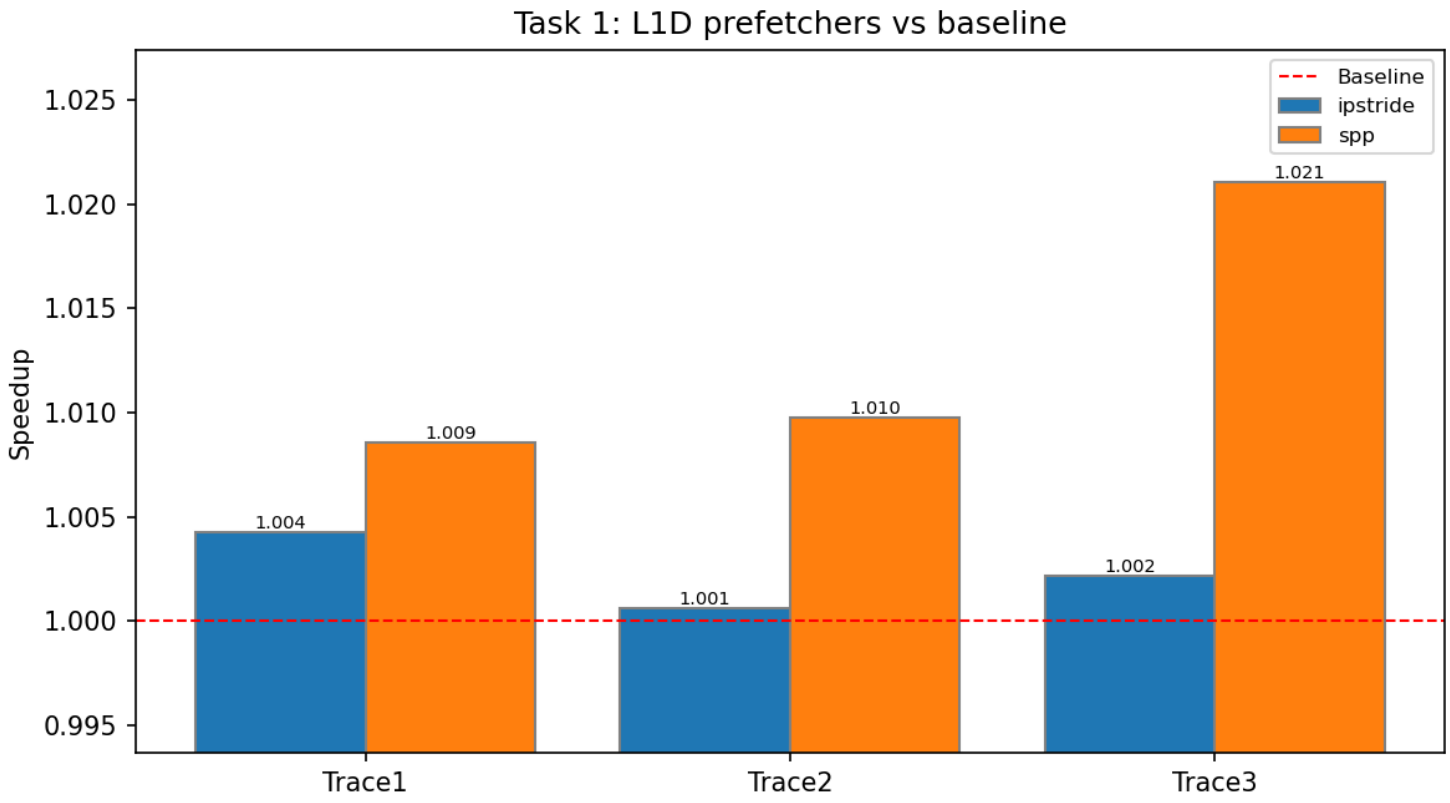
- **Why a window:** a cumulative ratio reacts too slowly to phase changes. A 256-prefetch window is short enough to track phases and long enough to be statistically stable.

Deliverables

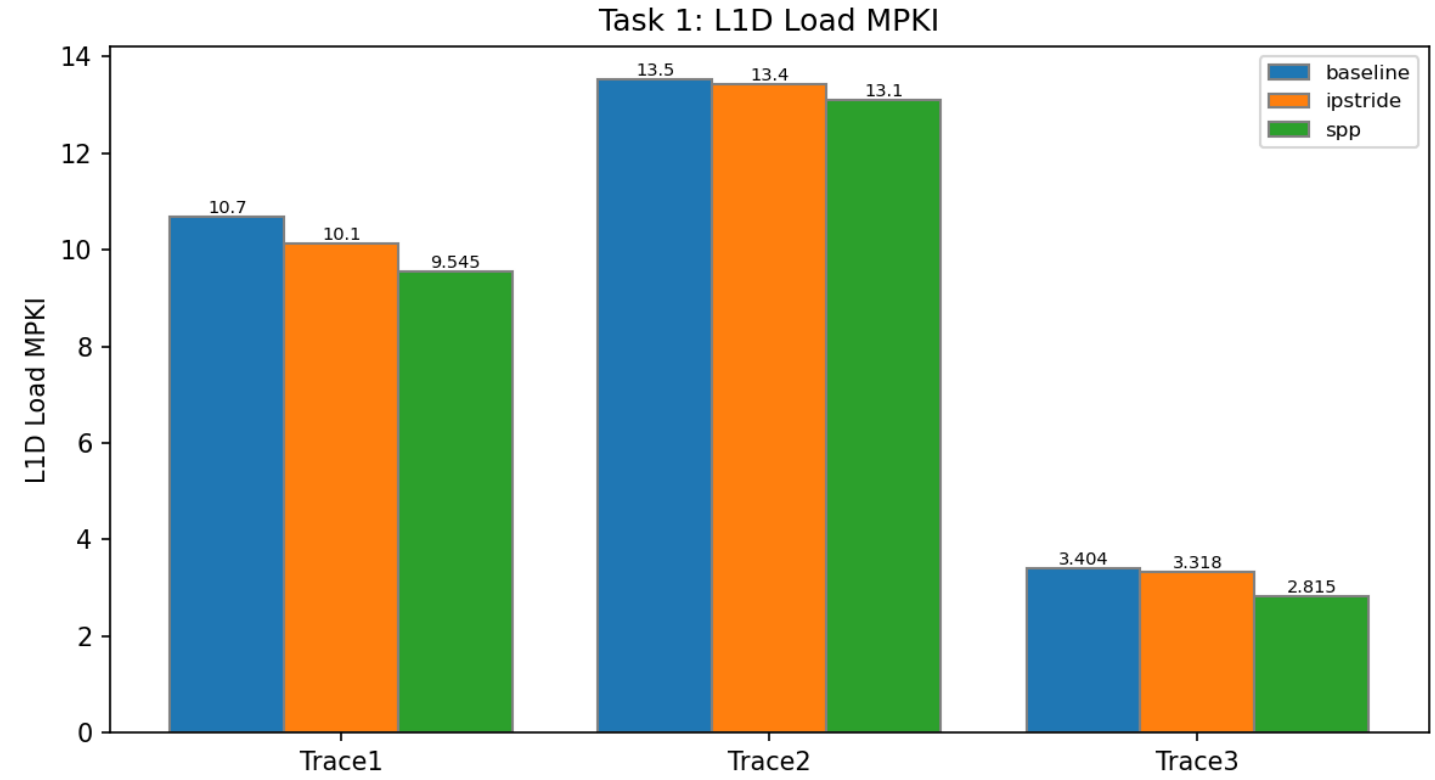
1. **Report the IPC, speedup, L1D Load MPKI, Prefetch accuracy, coverage values in tabular form, and plot them.**

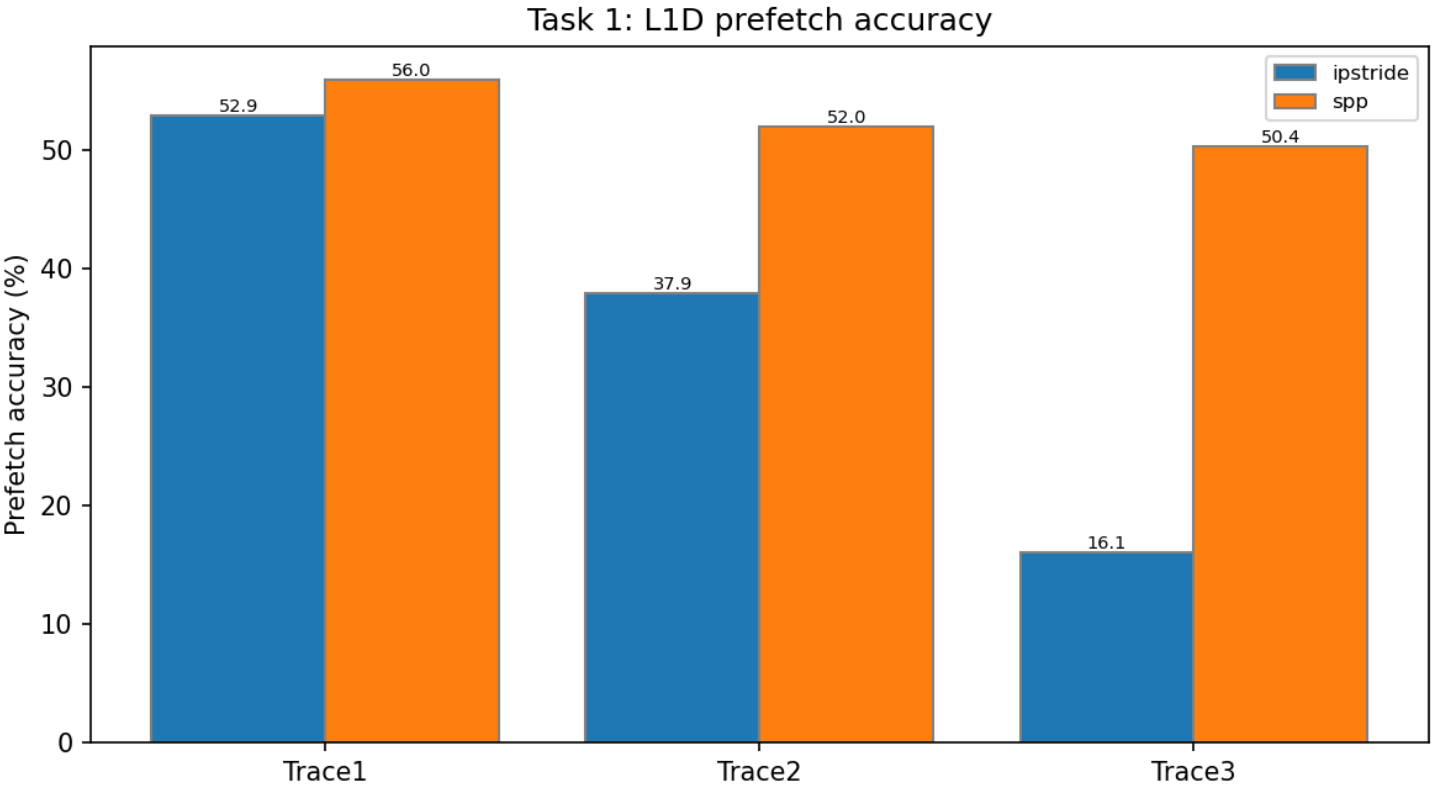
Trace	Config	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1	Baseline	0.2321	1.000	10.68	–	–
Trace1	IP-stride	0.2331	1.004	10.12	52.9	5.2
Trace1	SPP	0.2341	1.009	9.55	56.0	10.6
Trace2	Baseline	0.2500	1.000	13.54	–	–
Trace2	IP-stride	0.2501	1.001	13.44	37.9	0.7
Trace2	SPP	0.2524	1.010	13.09	52.0	3.3
Trace3	Baseline	0.3819	1.000	3.40	–	–
Trace3	IP-stride	0.3827	1.002	3.32	16.1	2.5
Trace3	SPP	0.3899	1.021	2.81	50.4	17.3

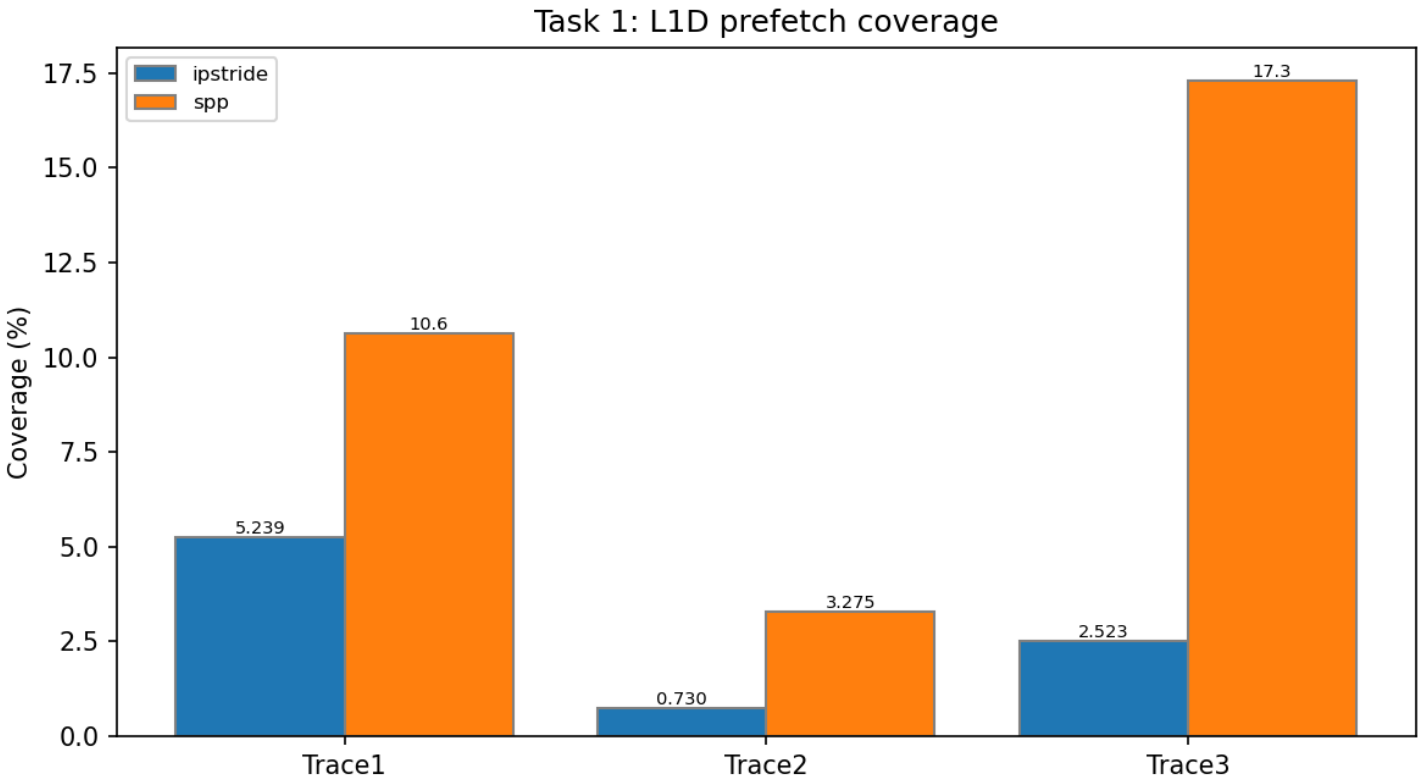
SPP is both more accurate and covers more misses than IP-stride on every trace. The largest gain is on Trace3, where coverage jumps from 2.5% to 17.3%: the likely explanation is that its loads follow



patterns a single remembered stride cannot capture but the signature path can (not verified per IP).


Task 1: L1D Load MPKI
L1D Load MPKI
baseline
ipstride
spp
10.7
10.1
9.545
13.5
13.4
13.1
3.404
3.318
2.815
Trace1
Trace2
Trace3


Task 1: L1D prefetch accuracy
Prefetch accuracy (%)
ipstride
spp
52.9
56.0
37.9
52.0
16.1
50.4
Trace1
Trace2
Trace3



2. What is the total storage overhead of your prefetcher? Provide an exact bit-level breakdown of every entry of your prefetching tables.

Storage overhead breakdown(bit-level)

Structure	Fields (bits)	Bits / entry	Entries	Total bits
IPT	valid 1 + tag 8 + last_page 8 + last_offset 6 + last_stride 7 + stride_conf 2 + sig 12	44	64	2,816
PT	next_stride 7 + conf 2	9	4,096	36,864
Throttle	window issued 9 + window useful 9 + degree 3	21	1	21
Total				**39,701 bits ≈ 4.85 KB**

The PT is 93% of the storage. A 10-bit signature (1,024 PT entries) would cut the total to about 1.5 KB at the cost of more signature collisions.

3. What and how did you decide the threshold for accuracy throttling?

Prefetch accuracy is measured over a sliding window of the last **256 prefetches issued to the lower-level cache (L2)**.

Accuracy is re-evaluated after every 256 issued prefetches.

Rationale for the Thresholds (25%, 50%, and 75%)

Design Decision	Explanation
50% — Break-even threshold	A useful prefetch can avoid a cache miss with an average L1D miss latency of approximately 140 cycles in our logs. A useless prefetch wastes bandwidth and may evict a useful cache line, causing additional misses. Below approximately 50% accuracy, prefetching may do more harm than good, so the prefetch degree should be reduced.
75% — High-confidence threshold	Accuracy of at least 75% indicates reliable predictions, allowing the prefetcher to increase its aggressiveness.
25% — Low-confidence threshold	Accuracy below 25% indicates that most prefetches are wasted. The prefetch degree is reduced to minimize bandwidth consumption and cache pollution.
Low hardware overhead	With a window of 256 prefetches, the thresholds correspond to 64, 128, and 192 useful prefetches. These comparisons can be implemented efficiently using counters, without requiring division.
Why a sliding window instead of a running total?	A running total reacts slowly when the program changes its memory-access behavior. A window of 256 prefetches provides a balance between adapting to phase changes and avoiding excessive reactions to short, unusually good or bad bursts.

4. Why did you choose these particular prefetch degrees for the different accuracy ranges (threshold)?

The prefetch degree determines how many future cache blocks are requested when the prefetcher makes a prediction. The degree is adjusted according to the measured accuracy.

Prefetch Accuracy	Prefetch Degree	Rationale
≥ 75%	6	Predictions are highly reliable, so the prefetcher looks further ahead to help hide the approximately 140-cycle memory latency. The degree is capped at 6 to limit diminishing returns, MSHR and queue pressure, and the uncertainty of increasingly distant predictions.
50% to 75%	4	Predictions are reasonably reliable. A moderate degree provides useful lookahead without excessive bandwidth consumption. This is also the starting degree.
25% to 50%	2	Accuracy is relatively low, so the degree is reduced to limit wasted bandwidth and cache pollution.
Below 25%	1	Predictions are very unreliable, but prefetching is not completely disabled. A nonzero degree allows the prefetcher to continue issuing requests and collecting feedback, enabling it to increase the degree again if accuracy improves.

Design Principle

The prefetch degree approximately halves as accuracy decreases across the lower bands. This makes the prefetcher less aggressive when predictions become less reliable and more aggressive when they become more accurate.

A minimum degree of 1 is important because setting the degree to 0 would stop issuing prefetches. If accuracy is measured only from newly issued prefetches, the feedback window would then stop receiving new samples, potentially preventing the prefetcher from recovering automatically.

Task 2a – Instruction-conscious RRIP (I-RRIP) at L2

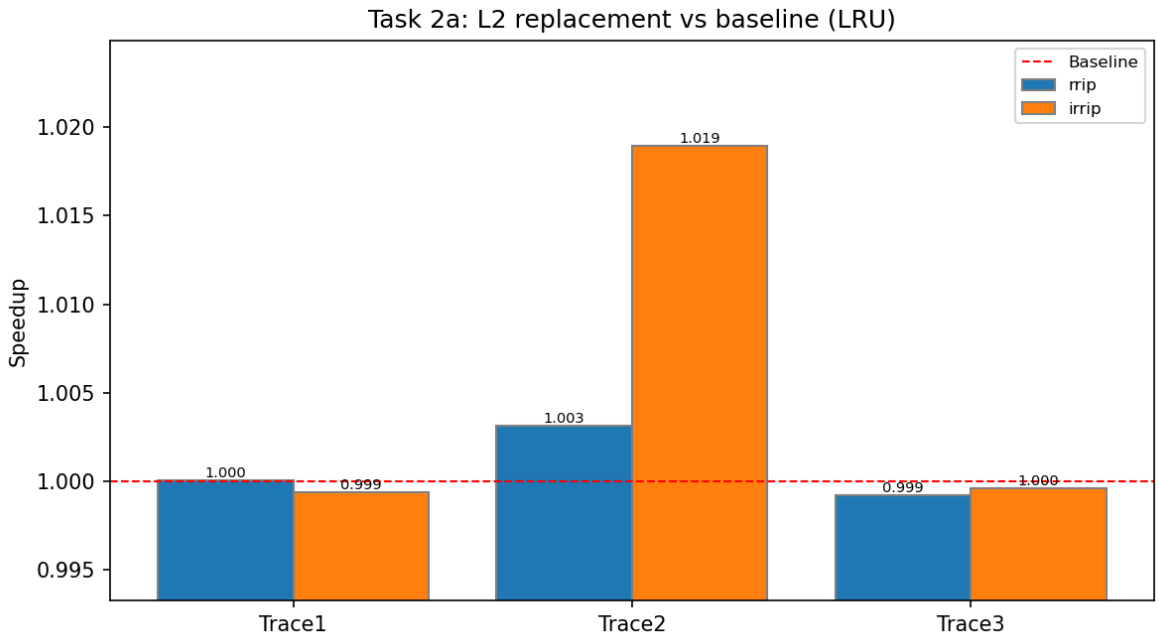
Deliverables

1. Report the IPC, speedup, and Load MPKI of L2C, L2 Instruction, and L2 Data in tabular form, and plot them.

Trace	Config	IPC	Speedup	L2C load MPKI	L2 instr MPKI	L2 data MPKI
Trace1	LRU (baseline)	0.2321	1.000	17.73	10.37	7.36
Trace1	RRIP	0.2321	1.000	17.66	10.58	7.07
Trace1	I-RRIP	0.2320	0.999	17.58	10.06	7.52
Trace2	LRU (baseline)	0.2500	1.000	28.55	19.84	8.71
Trace2	RRIP	0.2508	1.003	26.89	18.78	8.11
Trace2	I-RRIP	0.2547	1.019	24.17	15.17	9.00
Trace3	LRU (baseline)	0.3819	1.000	3.96	1.13	2.83
Trace3	RRIP	0.3816	0.999	4.04	1.20	2.84
Trace3	I-RRIP	0.3817	1.000	3.95	1.08	2.88

The trade-off is visible on every trace: I-RRIP lowers L2 instruction MPKI and raises L2 data MPKI. It only pays off on Trace2, where instruction MPKI falls by 4.7 (19.84 → 15.17) against a data MPKI rise of 0.29.

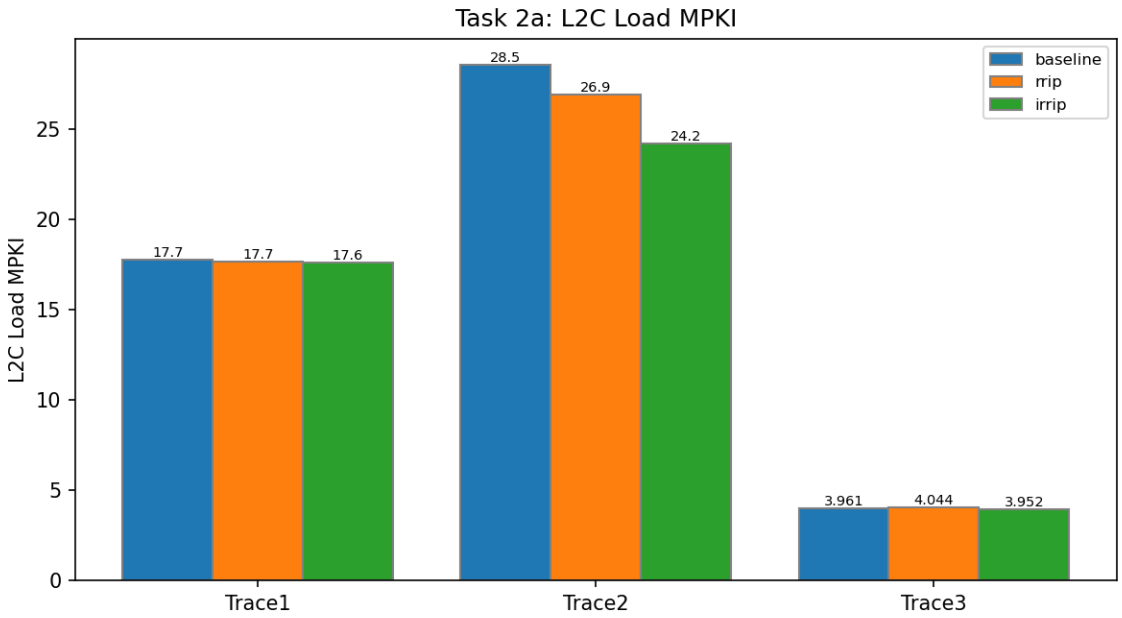
On Trace1 the instruction footprint is likely far larger than L2, so protection lowers instruction MPKI by

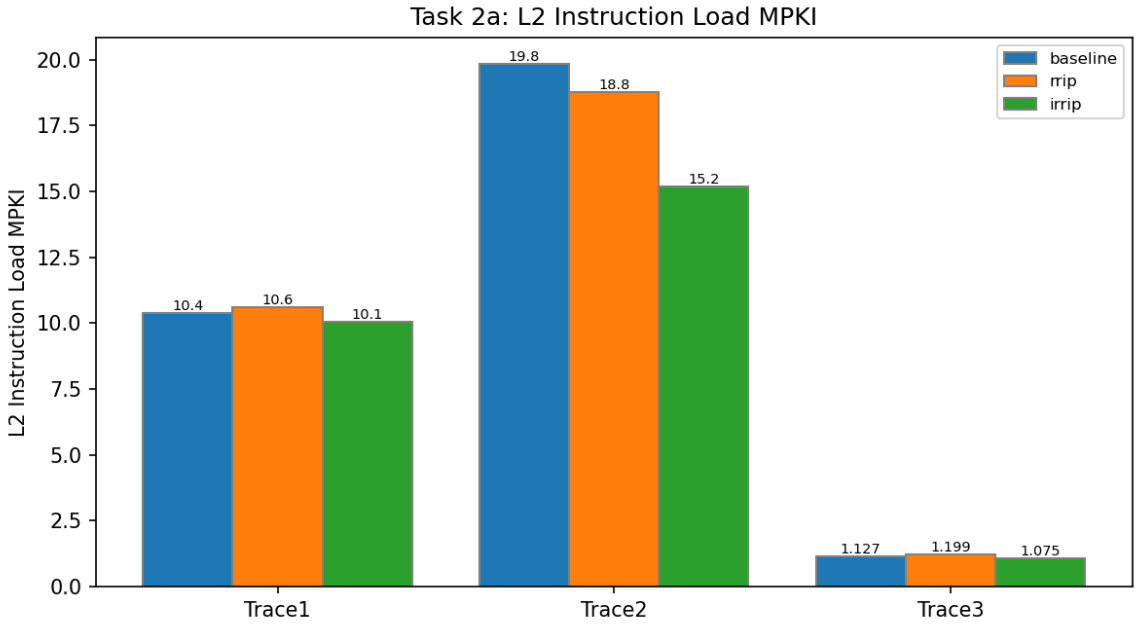


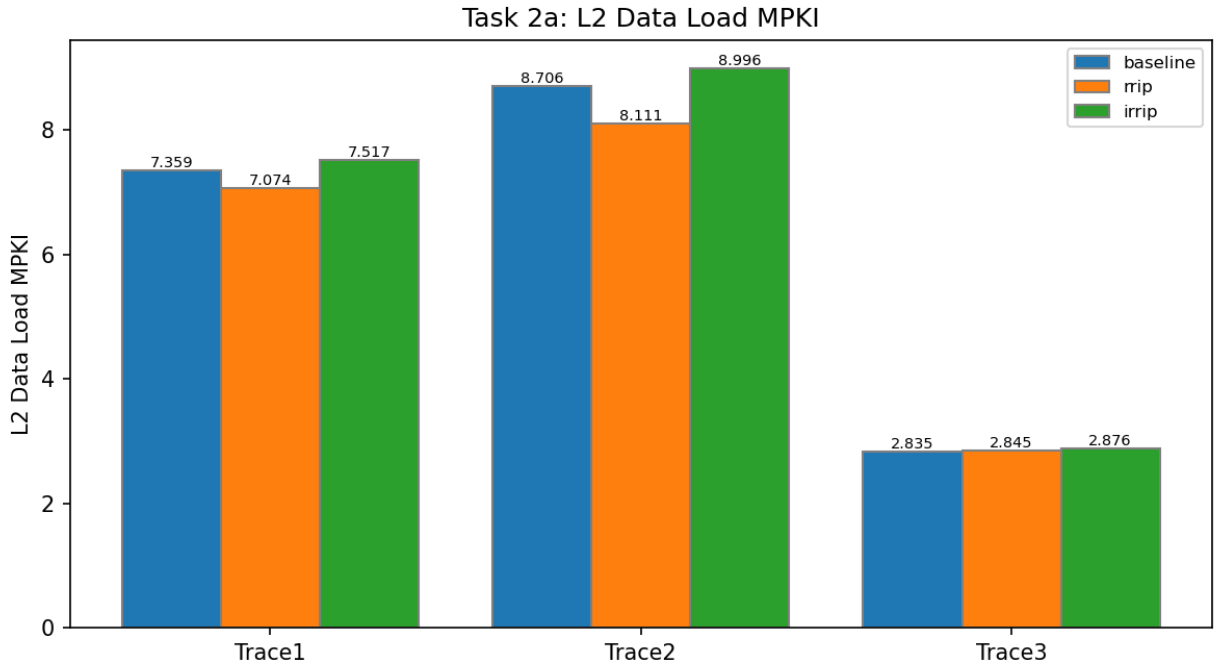
only 0.31 — not enough to offset the extra data misses. This trade-off is the motivation for Task 2b.

2. Explain the rationale behind modifying or not modifying insertion, promotion, and eviction logic. Defend why your way is the best. Make this section as descriptive and make your communication as clear as you can.

RRIP keeps a 2-bit re-reference prediction value (RRPV) per line: 0 = reuse soon, 3 = reuse far away, evict first. I-RRIP biases its three parts towards instruction lines, because an L2 instruction


Task 2a: L2C Load MPKI
L2C Load MPKI
baseline
rrip
irrip
17.7
17.7
17.6
28.5
26.9
24.2
3.961
4.044
3.952
Trace1
Trace2
Trace3


Task 2a: L2 Instruction Load MPKI
L2 Instruction Load MPKI
baseline
rrip
irrip
10.4
10.6
10.1
19.8
18.8
15.2
1.127
1.199
1.075
Trace1
Trace2
Trace3


Task 2a: L2 Data Load MPKI
L2 Data Load MPKI
baseline
rrip
irrip
7.359
7.074
7.517
8.706
8.111
8.996
2.835
2.845
2.876
Trace1
Trace2
Trace3

many data misses.

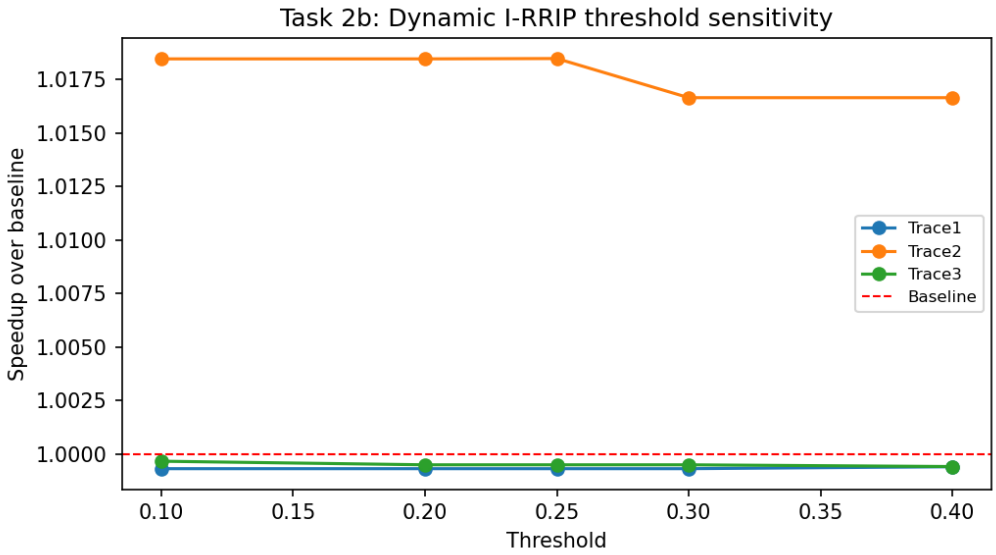
Part	RRIP	I-RRIP	Rationale
Insertion (miss)	RRPV 2 for all	instruction → 1, data → 2	Instructions get one extra aging step of protection. Not 0, so code that runs once (e.g. start-up) still ages out instead of squatting. Data keeps RRIP's "prove yourself" insertion so it is not penalised further.
Promotion (hit)	RRPV 0	RRPV 0 for both (unchanged)	A hit proves reuse regardless of type; demoting data hits would only add data misses.
Eviction	first line with RRPV 3	first **data** line with RRPV 3; else any line with RRPV 3; else age all	Among equally old lines, data goes first. The second pass is required: a set full of instruction lines would otherwise loop forever. Aging happens only when no line is at 3.

The instruction flag of a resident line is read from `current_set[i].instruction` (set by `fill_cache()`); the flag of the line being inserted or hit comes from the new `isInstruction` parameter. The parameter is needed because `update_replacement_state` runs before `fill_cache()` writes the new line's flag.

Task 2b – Dynamic I-RRIP (set dueling)

Deliverables

1. **Conduct a sensitivity study of your decision-making parameters; for example, if you are going with the threshold-based approach as mentioned in the hint, perform sensitivity analysis against 5 different values of the threshold.**



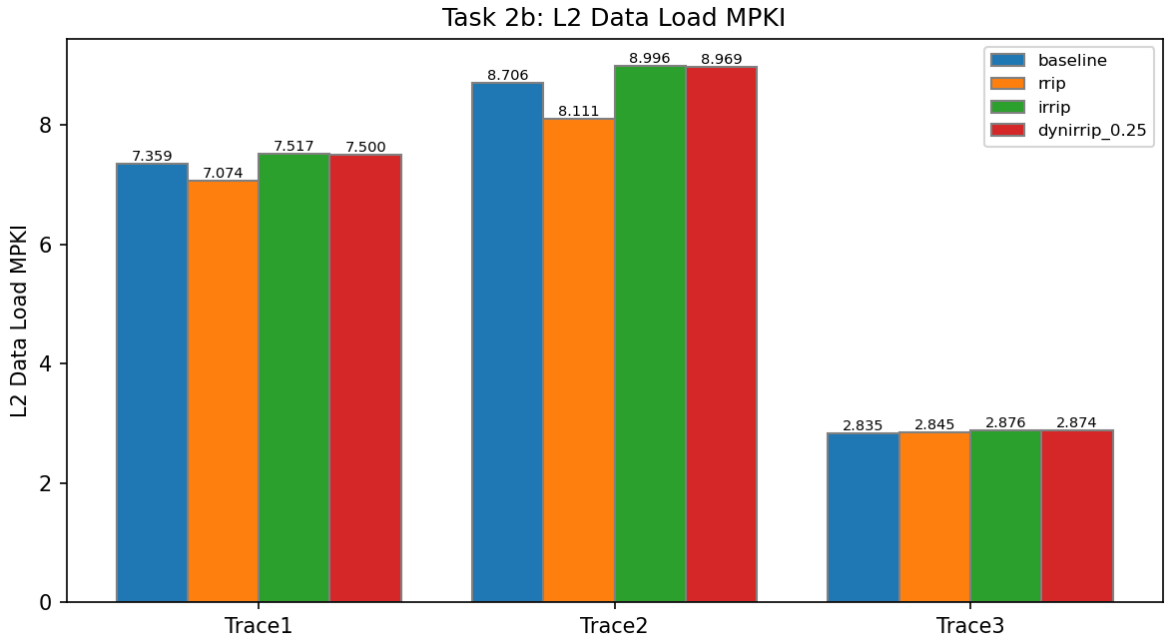
Threshold sensitivity

On Trace2, thresholds 0.10–0.25 give the best result; at 0.30 and above the policy falls back to RRIP in more epochs and loses 0.2%. Trace1 and Trace3 barely move at any threshold. Chosen threshold: **0.25**, tied best on Trace2 and within 0.01% of the best on the other traces.

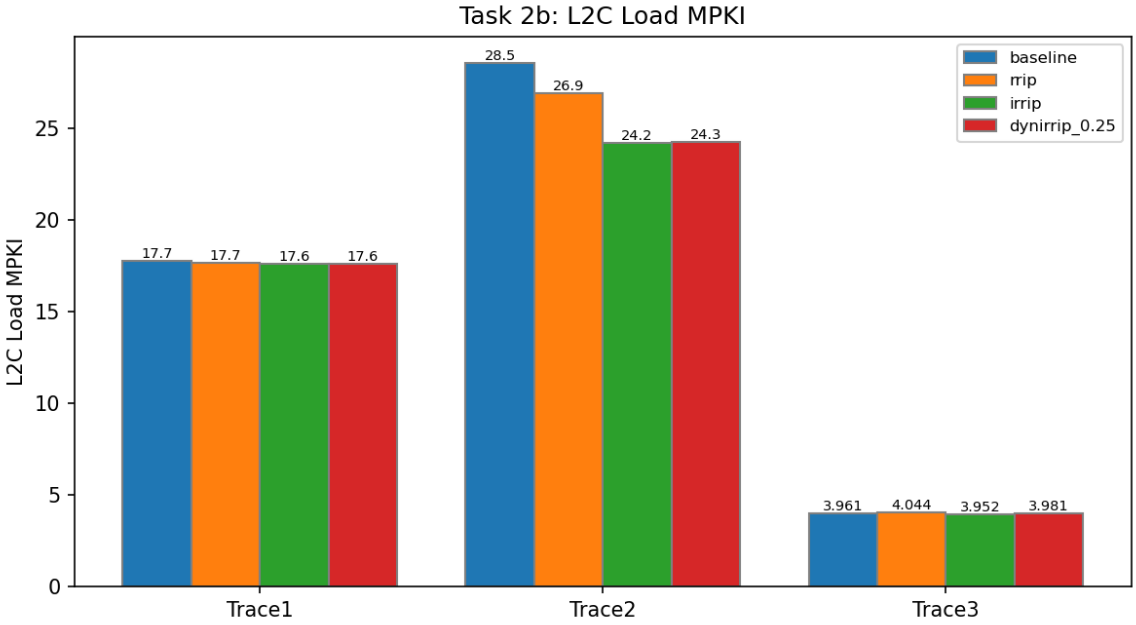
2. Report the IPC, speedup, and Load MPKI of L2C, L2 Instruction, and L2 Data in tabular form, and plot them.

Results at T = 0.25 =>

Trace	Config	IPC	Speedup	L2C load MPKI	L2 instr MPKI	L2 data MPKI
Trace1	Dyn I-RRIP	0.2320	0.999	17.58	10.08	7.50
Trace2	Dyn I-RRIP	0.2546	1.019	24.25	15.28	8.97
Trace3	Dyn I-RRIP	0.3817	1.000	3.98	1.11	2.87



3. Explain the rationale behind your number of leader sets and decision logic. Make this section as descriptive and make your communication as clear as you can.



Design

3.1 Set Dueling: The Idea

Set dueling dynamically decides whether instruction lines should receive preferential protection. A small group of **leader sets** measures the workload, while the remaining **follower sets** apply the selected policy. This provides workload adaptation with minimal hardware overhead.

3.2 Leader Set Selection

```
bool is_leader(uint32_t set) {
   return set % 32 == 0;
}
```

This selects sets 0, 32, 64, …, 992: **32 out of 1,024 sets (3.125%)**.

- **Representative sampling:** Even spacing samples different address regions. A contiguous group could be biased toward one code or data region.

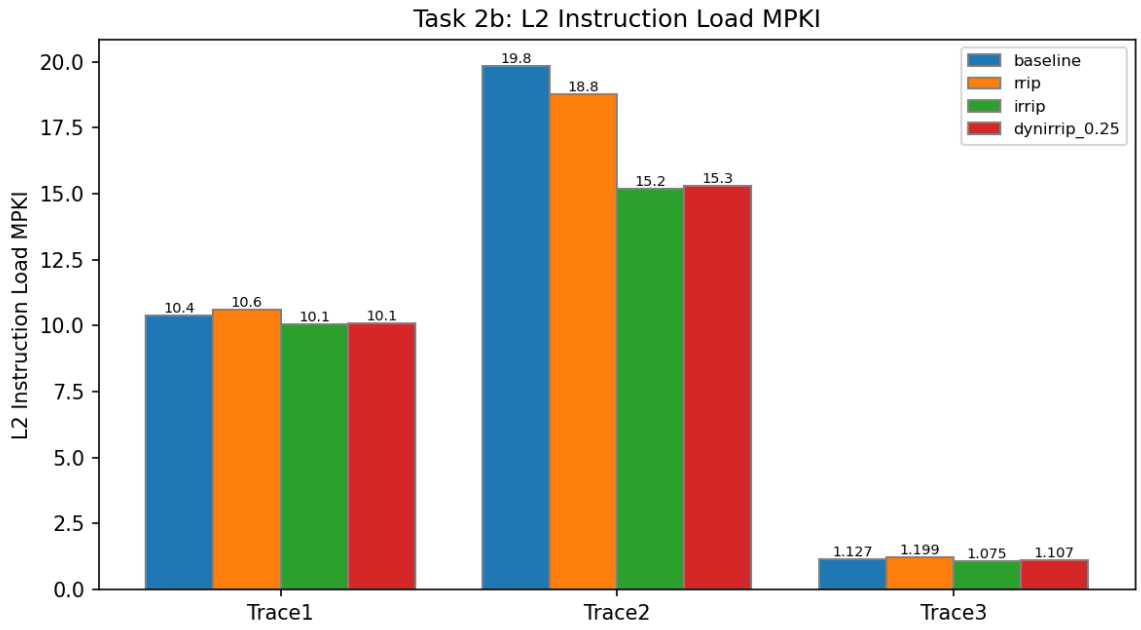
3.3 Why Leaders Always Use RRIP

Leader sets must measure instruction demand independently of the policy selected for followers. If leaders also used I-RRIP, protecting instruction lines could reduce instruction misses and lower the measured ratio, causing the followers to switch back to RRIP. The ratio could then rise again, producing oscillation.

Keeping leaders on plain RRIP avoids this feedback loop.

3.4 Counters

The replacement-state update function is called on L2 hits and fills. In leader sets, the counters are updated as follows:



Counter	Increment condition
`total_acc`	Every demand hit or fill
`total_miss`	Every fill
`instr_acc`	Every instruction hit or fill
`instr_miss`	Every instruction fill

Excluded: Writebacks are not demand accesses. Prefetches are also excluded because SPP issues data prefetches, which could bias the measurement according to prefetcher aggressiveness.

3.5 Decision Formula

[Ratio = (instruction access + instruction miss) / (total access + total miss)

Ratio > T => set IRRIP else set RRIP

Adding accesses and misses gives misses extra weight, making the ratio rise when instructions are both frequently accessed and frequently missed.

Threshold: T = 0.25, following the suggested value in the reference PDF. In the sensitivity study, Trace2 achieved its best speedup (1.0185) for T = 0.10–0.25, while T = 0.30–0.40 achieved 1.0166. Trace1 and Trace3 varied by less than 0.01%.

3.6 Epoch Size: 4,096 Leader Accesses

The four counters reset after every 4,096 leader accesses, allowing the policy to adapt to workload phase changes.

- **Stability:** For an estimated ratio p = 0.3, the approximate standard error is approximately 0.7 percentage points, reducing random decision changes compared with much smaller windows.
- **Responsiveness:** Smaller epochs react faster but are noisier; larger epochs are more stable but adapt slowly.
- **Hardware cost:** Four 13-bit counters and one use_irrip flag.

- **Observed limitation:** Trace3 had approximately 138K measured L2 load accesses, providing only around 4–6K accesses to the leader sets and therefore roughly one complete epoch. Trace2, with approximately 1.34M L2 load accesses, provided around ten decisions.

3.7 Why 32 Leader Sets?

- **Coverage:** The sets are evenly distributed across the cache.
- **Low overhead:** Only 3.125% of sets are leaders, leaving 96.875% as followers.
- **Too few leaders:** Epochs take longer to fill, reducing responsiveness and sampling coverage.
- **Too many leaders:** More cache capacity is permanently restricted to RRIP, even when I-RRIP would be better.

3.8 Policies

Operation	Leader sets	Followers using I-RRIP	Followers using RRIP
Insertion	All → RRPV 2	Instructions → 1; data → 2	All → 2
Promotion	→ 0	→ 0	→ 0
Eviction	**First RRPV-3 line**	**Prefer data RRPV-3 lines**	**First RRPV-3 line**

When the decision changes, the cache is not flushed. Existing lines retain their RRPVs, and the new policy applies to subsequent insertions and evictions.

3.9 Difference from Classic Set Dueling

Classic Dynamic Insertion Policy (DIP) uses two leader groups, one per policy, and a saturating PSEL counter to select the policy with fewer misses.

This design uses only RRIP leaders and selects I-RRIP according to the measured instruction-traffic ratio. It is simpler and cheaper, but measures how instruction-heavy the workload is rather than whether I-RRIP actually improves performance.

This limitation explains why I-RRIP can still be selected on Trace3 despite a 0.05% performance loss. A two-group design with direct performance comparison is a natural improvement.

Task 3 – Incorporating SPP with Dynamic I-RRIP

The combined design is the fastest configuration on Trace1 (1.0089) and Trace2 (1.0285), and ties SPP on Trace3 (1.0209 vs 1.0211, a 0.02% gap).

How the replacement policy handles prefetched lines

SPP's prefetches pass through L2, so L2 also holds a copy of each prefetched line, arriving with type == PREFETCH. Three changes to dyn_irrip.l2c_repl:

SPP's prefetches pass through L2, so L2 also holds a copy of each prefetched line, arriving with `type == PREFETCH`. Three changes to `dyn_irrip.l2c_repl`:

Part	Prefetch handling	Rationale
Insertion	prefetched line → RRPV 3	It is speculative (about 50% accuracy), and if it is useful the reuse hits in L1D, which L2 never sees. Inserting at 3 makes it the first candidate for eviction, so it cannot push out proven instruction or data lines.
Promotion	a prefetch hit leaves the RRPV unchanged	A prefetch request hitting a line shows that the prefetcher asked again, not that the program reused it. A later demand hit still promotes it to 0.
Set dueling	prefetches are not counted	SPP prefetches only data. Counting them inflates total_acc/total_miss but not the instruction counters, so the ratio would reflect SPP's aggressiveness rather than the program's needs. Example: 50 instruction accesses out of 100 demand accesses gives ratio 0.50 (I-RRIP); adding 300 data prefetches gives 50/400 = 0.125 and flips the decision to RRIP.

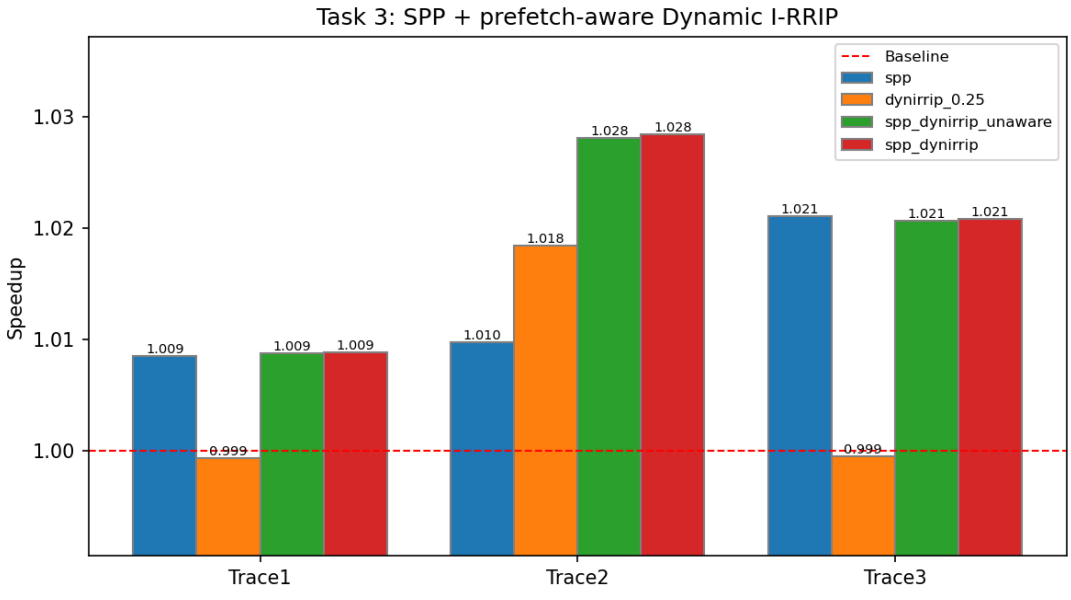
These rules apply in all sets, leaders included: prefetch handling is independent of the instruction-vs-data duel. With no prefetcher (Task 2b runs), `PREFETCH` never reaches L2, so Task 2b's behaviour is unchanged.

Performance Comparison

Trace	Config	IPC	Speedup	L1D MPKI	Accuracy (%)	Coverage (%)	L2C MPKI	L2 instr MPKI	L2 data MPKI
Trace1	SPP + Dyn I-RRIP	0.2342	1.0089	9.5508	55.5468	10.563	16.5691	10.043	6.5262
Trace2	SPP + Dyn I-RRIP	0.2571	1.0285	13.0975	51.5366	3.2485	23.7256	15.2208	8.5048
Trace3	SPP + Dyn I-RRIP	0.3898	1.0209	2.8207	49.9564	17.1256	3.1952	1.0721	2.1232

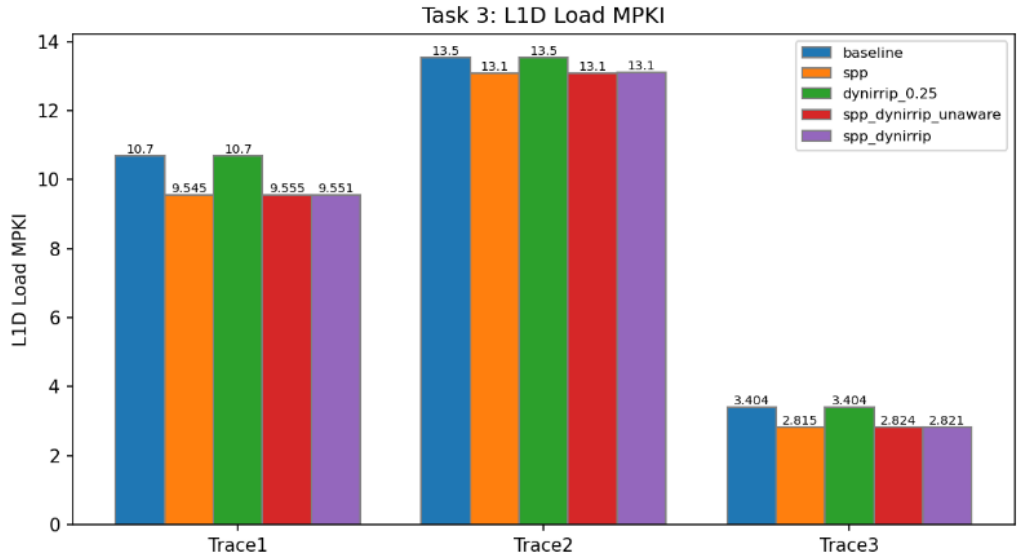
1. IPC and Speedup

- **Consistent Performance Wins:** The spp_dynrrip configuration consistently achieves the highest **ipc** and **speedup** values across all three traces compared to the standard baseline.
- **Outperforming Standalone SPP:**
  - For **Trace1**, IPC improves from 0.2341 (spp) to **0.2342** (spp_dynrrip).
  - For **Trace2**, IPC jumps noticeably from 0.2524 (spp) to **0.2571** (spp_dynrrip), yielding the highest speedup in that trace at **1.0285**.
  - For **Trace3**, it maintains a high speedup of **1.0209**, though slightly lower than the standalone spp (1.0211).



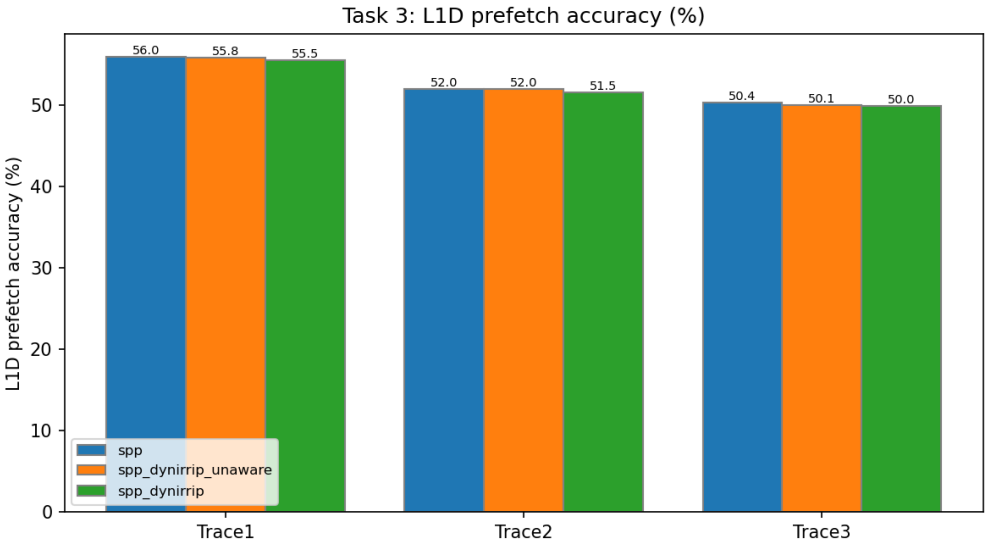
2. L1 Data Cache MPKI (l1d_load_mpki)

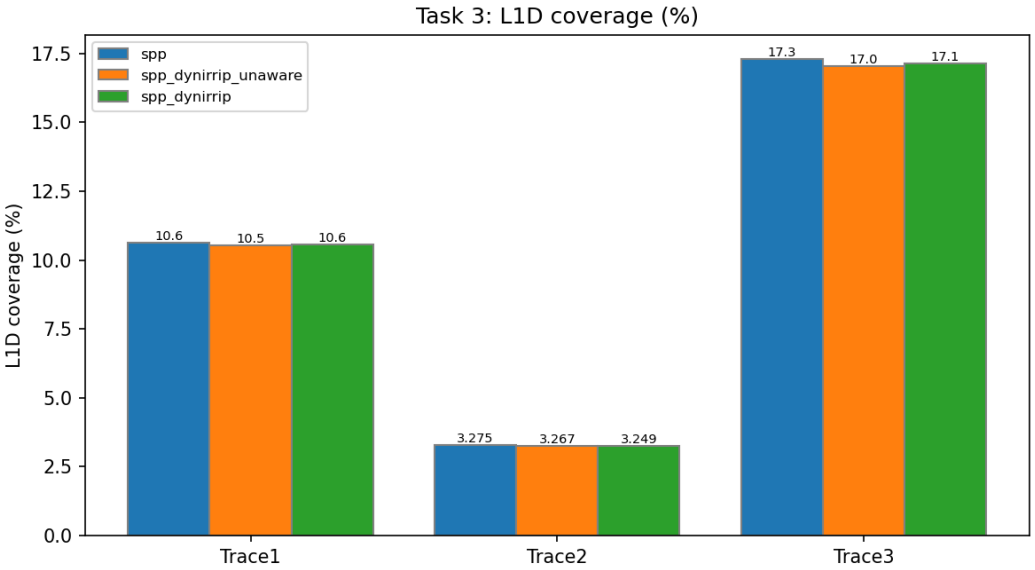
- **Significant Miss Reduction:** The prefetcher awareness drastically lowers L1 Data Cache misses per kilo-instruction compared to the baseline.
    - In **Trace3**, it drops the MPKI from 3.4036 down to **2.8207**.
- **Synergistic Improvement:** In **Trace1** and **Trace2**, spp_dynrrip achieves lower L1D MPKI numbers (**9.5508** and **13.0975** respectively) than standalone spp, demonstrating that the specialized insertion and promotion rules are successfully preventing useful demand lines from being prematurely evicted by prefetch bursts.



3. Accuracy and Coverage

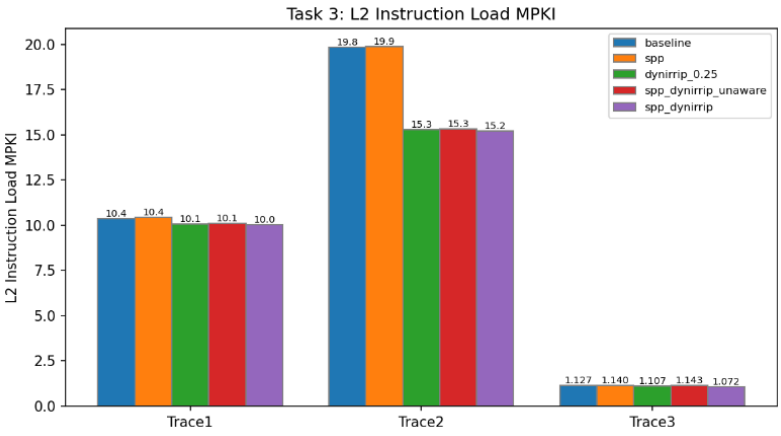
- **High Prefetch Accuracy:** spp_dynrrip maintains highly efficient accuracy metrics across the board, peaking at **55.5468%** accuracy in **Trace1**.
- **Trade-off in Trace 3:** In **Trace3**, while accuracy drops slightly compared to standalone spp (from 50.3706% down to **49.9564%**), the overall coverage remains strong at **17.1256%**, balancing aggressive prefetching with cache pollution awareness.

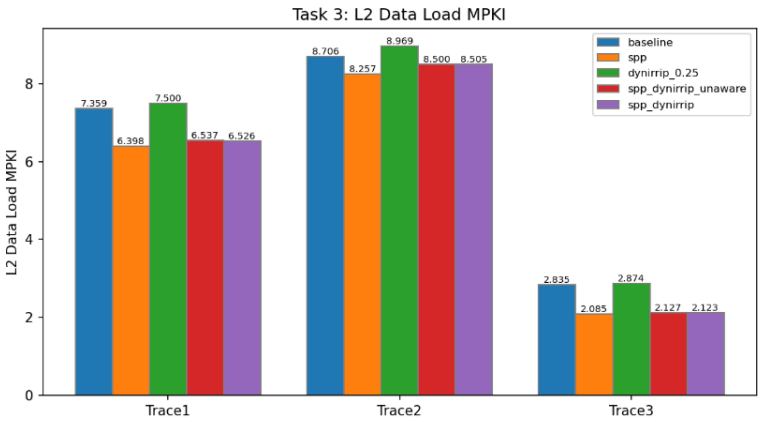




4. L2 Cache MPKI Metrics

- **L2 Load MPKI (l2c_load_mpki):** The aware scheme shows a clear drop in L2 load misses compared to the baseline across all traces. For example, in **Trace2**, misses fall from 28.5496 (baseline) to **23.7256** (spp_dynrrip). It also significantly beats the unaware version (spp_dynrrip_unaware) in Trace1 and Trace2, proving that set dueling and tracking rules prevent L2 trashing.
- **L2 Instruction and Data MPKI:**
  - **l2c_instr_mpki** remains fairly stable and low, indicating the changes do not negatively impact instruction streams.
  - **l2c_data_mpki** mirrors the load behavior, showing significant reductions across the board—most notably dropping from 8.7056 to **8.5048** in **Trace2** compared to the baseline.





Overall Observation

1. **Awareness Prevents Cache Pollution:** Making the cache replacement policy **aware** of the prefetcher is highly effective. Because prefetches enter with a lower priority (**RRPV 3**), they do not kick out critical data that the CPU actually needs right away.
2. **Beats Standalone Prefetching:** While a standard prefetcher (spp) reduces misses, it can clutter the cache with unused data. Combining it with dynamic RRIP acts as a filter—giving you the performance benefits of prefetching without the penalty of cache pollution.
3. **Trace Dependencies:**
    - In heavy-traffic workloads (**Traces 1 & 2**), coordinated awareness provides the ultimate boost to speed and cache efficiency.
    - In lighter workloads (**Trace 3**), even though accuracy drops near 50%, the RRPV 3 rule prevents performance drops by cleaning up inaccurate prefetches quickly.